

APEX — The AI / Architecture / Systems Frontier

Companion to the physics-layer research pass. For the IBM SkillsBuild AI Builders Challenge, May 31 2026 submission. Written for a 2-person team with a 12-day build window.

TL;DR

- **Keep frozen Granite TTM r2.1 as the *primary* forecaster, but stop using it bare. Wrap it in a three-track forecasting stack — (1) TTM with channel-mixing decoder fine-tuning + exogenous infusion ($\approx 5\%$ of target data, minutes on the RTX 4060) as the personalization head, (2) IBM Granite FlowState (the same Granite family, sampling-rate-invariant, #2 on GIFT-Eval at 9.1M parameters) as a complementary ensemble member, and (3) Amazon Chronos-2 as the zero-shot probabilistic-quantile baseline (21 quantiles, native covariates). Bare zero-shot TTM is not a defensible NeurIPS workshop story; this three-track stack is.**
- **Upgrade orchestration from "Langflow visual flow" to "Langflow demo over a LangGraph stateful backend, exposed via Model Context Protocol (MCP) through IBM's ContextForge gateway," with a tri-agent Agent-as-Judge critic loop (Physics-Critic, Pedagogy-Critic, Granite Guardian-Safety) and IBM's Mellea Instruct-Validate-Repair (IVR) as the per-paragraph faithfulness gate. This converts APEX from a pipeline into a real autonomous race-engineer agent and is the single biggest "shouldn't be possible" lever.**
- **Ship two demos a judge cannot ignore: a Granite-4.0-Nano-350M-ONNX WebGPU build that runs the post-race summary entirely in the driver's browser at the paddock with no internet, and an Activated-LoRA (aLoRA) "race-engineer intrinsic" served from vLLM with EAGLE-3 speculative decoding for 2-6 $\times$ wall-clock speedup. Both are recent (2025-2026) IBM-anchored capabilities a 2-person team can realistically land in 12 days, and together they make the project read as a systems paper, not a hackathon project.**

Key Findings (one paragraph each)

Forecasting is no longer a single-model decision. TTM is genuinely strong for its size (sub-1M parameters, NeurIPS '24), but as of May 2026 the GIFT-Eval frontier is held by **TimesFM-2.5**, **Chronos-2**, **Moirai-2.0**, **Toto-2**, and IBM's own **FlowState**, all of which post measurably better zero-shot accuracy. The committed "bare zero-shot TTM" choice is defensible only if you (a) lean on TTM's *channel-mixing fine-tuning* path that the team is currently ignoring, and (b) frame TTM as the *physics-friendly* member of an ensemble. Frozen-bare TTM on its own will not survive workshop peer review. (Hugging Face)

The committed Granite stack is using <30% of what IBM shipped in April 2026. Granite 4.1 introduced explicit Activated LoRA (aLoRA) adapters, Granite Switch (LoRA-multi-adapter

routing), **Granite Embedding R2** (ModernBERT, 8K context, top-of-class sub-500M retrieval), **TSPulse** (1M-param time-series classifier/anomaly model), **FlowState** (sampling-rate-invariant TSFM), and **Granite Guardian 4.1's Bring-Your-Own-Criteria (BYOC)** with hybrid thinking mode and best-of-N reward-model behavior that beats 70B-parameter reward models on JETTS. The team is using 8 tools but missing the 4 that would actually impress IBM judges.

Langflow is the wrong layer to stop at. Langflow is excellent for the demo video; it is not what a NeurIPS judge wants to see as the orchestration substrate. The 2026 frontier is **LangGraph (stateful, graph-based, cyclable) + MCP (Anthropic/Linux Foundation standard, 1-year-old, with more than 10,000 active public MCP servers covering everything from developer tools to Fortune 500 deployments per Anthropic's December 2025 Linux Foundation donation announcement) + IBM's ContextForge MCP Gateway**. Langflow can sit *on top* of LangGraph for the visual story while the substrate is auditable, testable, and certifiable.

Evaluation is what wins the workshop paper, not the model. Borrow the **LIPS 4-axis** ablation framework (accuracy, physical compliance, industrial readiness, OOD generalization) for the physics-projection layer, the **Sivalingam et al. (2026) LLM-as-Judge for Time-Series Explanations** protocol for coaching faithfulness, and the **DeepEval G-Eval custom rubric** template for the high-stakes coaching domain. Report max-violation, mean-violation, and violation-rate for the CvxpyLayer — the convention from differentiable-optimization-layer literature.

The "shouldn't be possible" moves are real, named, and current. In-browser Granite-4.0-Nano via WebGPU + Transformers.js (IBM's own demo Space, April 2026), Activated LoRA on vLLM (arXiv:2504.12397; multi-adapter serving paper arXiv:2512.17910), GEPA reflective prompt optimization (DSPy, ~30–53% gains in comparable high-stakes domains), EAGLE-3 speculative decoding (2–6× speedup), and Agent-as-Judge (90% agreement with human expert consensus vs. 70% for LLM-as-Judge per Zhuge et al., ICML 2025, arXiv:2410.10934). None of these require a research team; they require correct integration.

1. Forecasting Architecture Frontier

1.1 The current landscape (May 2026, GIFT-Eval-grounded)

Model	Params	GIFT-Eval posture	Key property	Relevance to APEX
TTM r2.1 (IBM)	<1M	Predecessor, pre-frontier	Tiny, CPU-runnable, channel-mixing decoder, exogenous & static-categorical infusion Hugging Face Hugging Face	Committed baseline; physics-friendly because tiny + decomposable
FlowState (IBM)	9.1M	#2 zero-shot, smallest in top-10	State-Space Model encoder + functional-basis decoder; sampling-rate invariant , continuous-time	Direct replacement candidate — solves the 50 Hz↔1 Hz aggregation problem natively
TimesFM-2.5 (Google)	200M	#1 zero-shot MASE/CRPS (Sep 2025)	Decoder-only, 16K context, half the size of v2.0	Strong baseline; univariate-only by default (XReg for covariates)
Chronos-2 (Amazon)	120M	90.7% average win rate on fev-bench (SQL metric) vs next-best TiRex at 80.8% per arXiv:2510.15821	T5-encoder, 21 quantiles , group-attention for multivariate + covariates Hugging Face	Best probabilistic forecaster — gives you the uncertainty bands
Moirai-2.0 (Salesforce)	~120M	Top-5 overall, top MASE among non-leaky	Decoder-only any-variate attention; native multivariate	Strongest for cross-channel coupling in telemetry
Toto-2 (Datadog)	4M–2.5B	Pareto frontier across all sizes	Trained on 2.36T tokens, Medium observability-focused, quantile head	Telemetry-shaped data is closer to observability than energy/weather
TSPulse (IBM)	1M	TSB-AD #1 (univariate & multivariate)	Dual-space (time+frequency) masked reconstruction; IBM classification,	Not a forecaster — add it for anomaly/classification heads

Model	Params	GIFT-Eval posture	Key property	Relevance to APEX
			anomaly, imputation, search	

1.2 Recommendation — CHANGE the framing, partially change the model

Confirm: Keep TTM r2.1 in the system. It is in the same Granite family as the rest of the stack, it is genuinely tiny, and the project explicitly needs to showcase IBM Granite for the challenge. Bare zero-shot remains its baseline use.

Recommend changing: Stop calling it "the forecaster." Build a **three-track forecasting head** and ensemble the outputs into the CvxpyLayer:

1. Track A — TTM r2.1 with channel-mixing decoder fine-tuning + exogenous infusion.

This is documented in `granite-tsfm`. Set `decoder_mode="mix_channel"` and pass `prediction_channel_indices` for the target (speed, lateral_g) and `exogenous_channel_indices` for known-future controls (throttle, brake, steer). Freeze the backbone, fine-tune the head only — minutes on the RTX 4060 with ~5% of the target data. This is the *personalization* head. (IBM) (Hugging Face)

2. Track B — Granite FlowState (r1). Same Granite family, Apache 2.0, drop-in via `granite-tsfm`. It is **sampling-rate-invariant by construction** — the most elegant answer to the "50 Hz raw telemetry vs TTM's hourly/minute support envelope" problem. Use FlowState for any 50 Hz channels you cannot polyphase-downsample cleanly. NeurIPS 2025 paper, IBM Research authors — perfect citation hygiene.

3. Track C — Chronos-2 quantile head. 21 quantiles, native covariate support, runs at ~300 series/sec on a single A10G (so trivially fast on the laptops). Use the 0.1 / 0.5 / 0.9 quantiles to draw the *uncertainty corridor* the report ships to the driver. This is the channel that lets you say "we forecast 1.7G lateral, 80% probability the true value lands between 1.5G and 1.9G" — which is what "next-session forecast" actually means. (Hugging Face)

Combine the three with a small learned ensembler (or, for honesty, a fixed weighted mean) and feed the *median + uncertainty quantiles* into the CvxpyLayer.

1.3 The "kinetic hallucination" framing is good but undersells the fix

Channel-mixing decoder fine-tuning in TTM already mitigates some kinematic incoherence by letting the decoder see inter-channel correlations. Note this explicitly in the paper: **kinetic hallucination is not solved by the projection alone; it is reduced by (a) channel-mixing, (b) exogenous infusion of known-future controls, and (c) the convex projection**. This is a cleaner story than "TTM hallucinates so we add a QP." (Hugging Face)

1.4 Confirm vs. change — forecasting

Element	Verdict	Why
Use Granite TTM r2.1 at all	CONFIRM	Family fit, tiny, channel-mixing exists
Bare zero-shot only	CHANGE	Add channel-mix fine-tune + exogenous infusion
Single-model forecasting	CHANGE	Add FlowState + Chronos-2 as ensemble members
1 Hz aggregation as the only frequency strategy	CHANGE	FlowState handles sampling-rate variation natively
Point forecast only	CHANGE	Chronos-2 gives 21 quantiles — ship uncertainty bands

2. The Full IBM Granite Stack at Maximum

The committed 8 tools are good. The maximum, given the 12-day window, is **12 tools** by adding four high-leverage IBM components that are already Apache 2.0 and pip-installable.

2.1 What the committed tools should actually do

Tool	Committed use	Maximal use
Granite-Docling 258M	Parse the COA PDF	Parse the COA PDF into DocTags (IBM's structured layout markup), then route DocTags into the Granite 4.1 tokenizer vocabulary (IBM has stated DocTags samples are being added to future Granite training data and to the Granite tokenizer vocabulary) so the LLM has <i>structured</i> COA fields, not a flat text blob. Use vLLM for ~0.35 s/page on A100; <u>Ollama</u> on the 4060, fall back to MLX/CPU.
Granite Vision 4.1 (4B)	Backup PDF reader	Use it for chart extraction from any uploaded session screenshot (MoTeC traces, Track Attack, AiM RaceStudio). Granite Vision 4.1 holds the top score across 7 IBM-listed benchmarks (chart, table, KVP extraction) and IBM reports a score of 86.5 on table extraction vs Claude Opus 4.6 at 83.8. This unlocks a second data modality without writing a parser.

Tool	Committed use	Maximal use
Granite TimeSeries TTM r2.1	Zero-shot forecaster	Channel-mix fine-tuned head (§1)
Granite 4.1 8B Instruct	Coaching report writer	Dense 8B that "consistently matches or outperforms the Granite 4.0 32B Mixture-of-Experts model" (IBM) (IBM Research blog) on tool calling (BFCL V3 68.3) and instruction following — use it as the planner-executor agent with native function calling, not a one-shot report generator.
Granite Guardian 4.1 8B	Audit a textual violation log	Bring-Your-Own-Criteria mode with custom criteria like <i>"every numerical claim in the coaching paragraph appears within $\pm 5\%$ of the corresponding CvxpyLayer output"</i> . Use hybrid thinking mode: <code><no-think></code> for the per-paragraph gate (low latency), <code><think></code> for the final safety verdict. (Runpod) Also use Guardian as a reward model for best-of-N sampling : per IBM's model card, "Granite Guardian 4.1 8B achieves the highest overall score (70.29) among all tested reward models, outperforming models up to 70B parameters" on the JETTS benchmark.
Langflow	Visible orchestration	Keep as the demo visualization , but route the actual graph through LangGraph (§3) and expose it as an MCP server (Langflow does this natively) (ZenML) so a judge can connect Claude Desktop / Cursor to the APEX flow and call it as a tool.
Docling library	PDF→Markdown	Combine with Granite-Docling for the ensemble pipeline; expose <code>docling-mcp-server</code> from IBM's MCP collection as a callable tool.
IBM Bob	Build accelerator	Use for the codebase scaffolding only; do not put it in the architecture diagram.

2.2 The four missing IBM components to add

1. **Granite Embedding R2 (English, small + full)**. ModernBERT-based, 47M or 149M parameters, 8192-token context, Apache 2.0, top-of-class sub-500M retrieval on MTEB. This is the embedding model for the RAG layer (§3b). Use the **149M full model** for the corpus encoder and the **47M small model** for query encoding — IBM also ships a cross-encoder reranker as part of R2 that you can stack for high-precision reranking.

- 2. **Granite TSPulse r1 (1M params)**. TSB-AD #1 anomaly detector; IBM reports "+20% on the TSB-AD anomaly detection leaderboard" — use it as the **anomaly head** that flags "this corner is unusual for you" before the LLM writes coaching. Same family, GPU-free inference. Adds a genuinely useful capability without adding a non-Granite dependency. [IBM](#) [Hugging Face](#)
- 3. **Granite FlowState r1 (9.1M params)**. Ensemble member for forecasting (§1). NeurIPS 2025 workshop paper — a free citation that strengthens your paper because you can credibly say "the same SSM architecture that placed #2 on GIFT-Eval is in our system."
- 4. **Granite 4.0 Nano (350M, ONNX-web)**. The browser deployment (§5). IBM's official [ibm-granite/Granite-4.0-Nano-WebGPU](#) Space exists; the [onnx-community/granite-4.0-350m-ONNX-web](#) weights are public. [Hugging Face](#)

2.3 Optional / stretch (post-Day-8 if time)

- **Granite Switch 4.1 (preview)** — dynamic LoRA-adapter switching via control tokens at inference time. Bundles 12 IBM adapters (RAG, guardian-core, factuality detection/correction, policy guardrails). If you swap Granite 4.1 8B for Granite Switch 4.1 3B-preview, you get the full Guardian Library in one checkpoint. [Hugging Face](#) [Hugging Face](#)
- **Granite Speech 4.1 (2B)** — if the team has 2 days spare, voice-narrated coaching playback. IBM reports a 5.33% word error rate on enterprise benchmarks. [Aimadetools](#)
- **granite-tsfm** — already a dependency; expose [TimeSeriesAnomalyDetectionPipeline](#) from this library to drive TSPulse.

2.4 Confirm vs. change — Granite stack

Element	Verdict	Why
Use Granite stack at all	CONFIRM	It's an IBM challenge
8 Granite tools	CHANGE → 12	Add Embedding R2, TSPulse, FlowState, Granite 4.0 Nano WebGPU
Granite Guardian as a textual auditor only	CHANGE	Use BYOC for numerical-claim faithfulness + use as reward model for best-of-N
Granite Vision 4.1 as a backup	CHANGE	Use it as the chart/table reader for MoTeC screenshots
IBM Bob in the architecture	CHANGE	Use as build tool only; remove from system-of-record diagram

3. Agentic Orchestration Frontier

3.1 Where the field is, May 2026

The 2025–2026 frontier is **graph-based stateful runtimes (LangGraph, BeeAI, AutoGen) + Model Context Protocol (MCP) for tool federation + dedicated guardian/judge models**. Pure visual orchestrators (Langflow, Flowise, n8n) are now considered prototyping layers, not production substrates.

Key 2026 facts:

- MCP turned 1 year old in November 2025, is governed by the Linux Foundation since December 2025, and has **more than 10,000 active public MCP servers, covering everything from developer tools to Fortune 500 deployments** per Anthropic's official Linux Foundation donation announcement. The November 2025 spec added server-side agent loops and tool calling in sampling — i.e., agentic behavior is now native to MCP.
[Modelcontextprotocol](#)
- IBM ships **ContextForge MCP Gateway** — "An AI Gateway, registry, and proxy that sits in front of any MCP, A2A, or REST/gRPC APIs, exposing a unified endpoint with centralized discovery, guardrails and management" (IBM/mcp-context-forge GitHub). Apache 2.0.
[GitHub](#)
- BeeAI (IBM) is the IBM-blessed multi-agent framework; LangGraph is the de facto external standard.
- The **Agent-as-Judge** paradigm: per Zhuge et al. (ICML 2025, arXiv:2410.10934), "in code-generation experiments, Agent-as-a-Judge agreed with human expert evaluations about 90% of the time—substantially outperforming the 70% agreement rate of previous LLM-as-a-Judge methods."

3.2 The maximal APEX agent architecture

Run two layers:

Layer 1 — Demo / Visualization: Langflow. Keep the visible flow because it is *the* IBM-blessed visual differentiator and the judges will love it on screen. Export the Langflow project as an MCP server (Langflow does this natively) and treat it as the *façade*.

Layer 2 — Substrate: LangGraph state machine, exposed over MCP. A state machine with these named nodes:

1. [ingest](#) — Granite-Docling parses COA, Granite Vision reads any chart screenshots, telemetry CSV → parquet.
2. [decompose](#) — polyphase decomposition + mini-sector tensor build (from the prior physics pass).

3. `forecast` — ensemble of TTM (channel-mix fine-tuned) + FlowState + Chronos-2 quantiles.
4. `anomaly` — TSPulse flags unusual corners.
5. `project` — Stage 1 CvxpyLayer + Stage 2 non-convex feasibility filter (already designed).
6. `retrieve` — Granite Embedding R2 retrieves from the racing-line/setup corpus (§3b).
7. `plan` — Granite 4.1 8B Instruct planner picks an outline.
8. `draft` — same model writes the corner-by-corner sections.
9. `critique` — **tri-agent panel**: (a) Physics-Critic re-runs the numbers against the CvxpyLayer output, (b) Pedagogy-Critic scores explanation quality on a fixed rubric, (c) Granite Guardian 4.1 in BYOC mode checks per-paragraph numerical faithfulness and per-call function-call hallucination.
10. `repair` — IBM **Mellea Instruct-Validate-Repair (IVR) loop**: per Mellea docs, "When the model's output fails a requirement, Mellea can retry the generation with feedback ... set a `loop_budget` and the probability of the output satisfying your requirements approaches 1 as budget increases. Requirements can be simple string constraints, Python validation functions, or powerful model-based validators like IBM Granite Guardian." Use `loop_budget=3`. `Mellea`
11. `verify` — final Guardian safety verdict; emit the stamp.

Tool calling: all numerical tools (CvxpyLayer, polyphase, friction-ellipse) are exposed as MCP tools via ContextForge so the planner can invoke them by name, with audit traces in OpenTelemetry. This single move makes APEX *agentic* rather than *pipelined* — a critical narrative shift for judges.

3.3 Why this beats LangChain-only / Langflow-only

Langflow alone gives you a pretty graph and no statefulness. LangGraph alone gives you statefulness and no visual story. MCP alone gives you tool standardization. The three combined give you all of: visual demo + production substrate + standardized tool federation + observability.

3.4 Confirm vs. change — orchestration

Element	Verdict	Why
Langflow for "visible orchestration"	CONFIRM as demo layer	IBM-promoted, visually compelling
Langflow as the runtime	CHANGE	Use LangGraph + MCP as substrate
One-shot LLM call to write the report	CHANGE	Plan→Draft→Critique→Repair with Mellea IVR

Element	Verdict	Why
Single safety check at the end	CHANGE	Per-paragraph Guardian gate + tri-agent critic
Tools wired as Python imports	CHANGE	Expose as MCP tools through ContextForge

3b. Retrieval and Grounding

3b.1 Yes — APEX should be RAG-grounded

Coaching prose without grounding is exactly the kind of free-text where LLMs hallucinate confident-sounding nonsense. The driver is a *user with a safety contract*. Ground every claim.

3b.2 The maximal grounding architecture

Corpus (~200–500 documents, curated, all open-license):

- Racing-line theory: *Going Faster!* (Skip Barber), Bondurant, *Optimum Drive* (Paul Gerrard), *Drive to Win* (Carroll Smith), *The Perfect Corner* (Adam Brouillard) — pages and excerpts where licensing permits, otherwise extracted principles
- Vehicle setup: Carroll Smith's *Tune to Win*, *Engineer to Win*; Milliken & Milliken *Race Car Vehicle Dynamics* (citation-only where copyright)
- FIA Sporting Code 2026, Appendix L Chapter III (driver equipment), FIA Disability & Accessibility Commission documents
- Manufacturer setup guides for popular grassroots cars (Mazda Miata, Honda Civic, Caterham 7)
- The driver's own historical sessions (per-driver shard, encrypted at rest)

Ingestion: Docling + Granite-Docling-258M → DocTags → Granite Embedding R2 (full 149M) → chunk at 8K tokens with semantic boundaries.

Vector store: lightweight LanceDB or Qdrant local-only (no managed service). Both are Apache 2.0, embeddable, and run on the 4060.

Query path:

1. Granite Embedding R2 small (47M) for fast query encoding
2. Hybrid retrieval: dense (Granite R2) + sparse (BM25 via `rank-bm25`)
3. **Cross-encoder rerank** with the Granite R2 reranker — this is the precision step
4. Pass top-k as context to the planner

Knowledge-graph layer (stretch goal, post-Day-9): Build a small driving-physics ontology (corner-types × tire-state × control-input nodes) and use it as a routing layer over the corpus. This is genuinely impressive in a workshop talk and feasible because the ontology is small (<100 nodes).

Per-driver memory: the driver's prior sessions live in a *per-driver collection* — RAG-as-memory pattern. Granite 4.1's 512K context can technically swallow a whole season raw, but retrieval is cheaper and more honest. [Aimadetools](#)

3b.3 Confirm vs. change — grounding

The committed design did not specify a RAG layer. **RECOMMEND ADDING** a Granite-Embedding-R2-based RAG layer with cross-encoder reranking, hybrid dense+sparse retrieval, and a per-driver memory collection. This is the single biggest defense against the LLM inventing tire pressures and shock settings.

4. Evaluation, Benchmarking, and Validation Methodology

This is the section that converts the hackathon project into a NeurIPS workshop paper.

4.1 Adopt the LIPS 4-axis ablation framework

From Vaitl et al. (arXiv:2509.19233): "Using our custom benchmarking pipeline for hybrid models called LIPS, we evaluate these models across four dimensions: accuracy, physical compliance, industrial readiness, and out-of-distribution generalization." [arxiv](#)

Build your ablation table on exactly these four axes:

Axis	Metric	What it answers
Accuracy	MASE, CRPS, WQL per channel	Is the forecast right?
Physical compliance	Max constraint violation, mean violation, violation rate per lap, fraction of forecasts requiring projection	Does the system hallucinate kinematically?
Industrial readiness	End-to-end p50/p95/p99 latency, peak RAM, model size, cost per coaching report	Can it ship?
OOD generalization	Held-out track × car-class × COA-class	Does it work for a driver/car/track it's never seen?

The DFWLayer convention (arXiv:2308.10806) is to report **max violation, not standard deviation** — adopt that convention. Also adopt **constraint-satisfaction regret** (cost rate, à la Ray et al. OpenAI safety benchmarking) and **EMCC (Expected Maximum Constraint Cost)** to quantify *severity* of violations, not only frequency (Eckel et al., 2024). (arXiv)

4.2 Ablation table (mandatory)

Four rows, all four axes:

1. **Forecaster-only** (TTM zero-shot, no projection)
2. **Forecaster + soft penalty** (Lagrangian loss term, no hard projection)
3. **Forecaster + Stage-1 CvxpyLayer** (your contribution)
4. **Forecaster + Stage-1 + Stage-2 non-convex feasibility filter** (full APEX)

Plus a *baseline-of-baselines* row: replace the forecaster with a seasonal-naïve predictor and run rows 1-4 again. This isolates the contribution of TTM vs. the projection. Cite Dalal et al. (2018) safety-layer QP as the conceptual precedent for the CvxpyLayer, via arXiv:2505.17342: "Dalal et al. (2018) introduced a safety layer that solves a quadratic program (QP) in continuous action spaces to minimally adjust the action such that predicted next state stays within safety bounds. This method guaranteed zero constraint violations during training on those tasks." (arXiv)

4.3 Coaching faithfulness evaluation

The closest published evaluation problem is **Sivalingam et al., "LLM-as-a-Judge for Time Series Explanations"** (arXiv:2604.02118), which states the problem exactly: "Evaluating the factual correctness of natural language explanations generated by LLMs and grounded in time-series data remains an open challenge ... there is currently no general-purpose method for directly verifying whether an explanation is faithful to the underlying time-series data without predefined references or task-specific rules." Cite this in related-work and present your evaluation as a contribution. (arXiv)

Three-probe faithfulness protocol (from arXiv:2603.13988): "Our study consists of three perturbation-based probes: (1) causal ablation, testing whether stated chain-of-thought (CoT) reasoning causally influences predictions; (2) positional bias, examining whether models create post-hoc justifications for answers driven by input positioning; and (3) hint injection, testing susceptibility to external suggestions." Adapt directly: (arxiv)

1. **Causal ablation:** scrub the numerical context, regenerate the report — does the prose change?
2. **Positional bias:** shuffle the corner ordering — does the same corner get the same coaching?
3. **Hint injection:** inject a wrong physics hint — does the model defer or correct?

Custom G-Eval rubric (DeepEval, drop-in template from Confident AI): define a

`RacingCoachingFaithfulness` metric whose evaluation steps are (1) extract numerical claims from the actual output, (2) verify each against the TTM forecast + CvxpyLayer-projected values + retrieved corpus, (3) identify contradictions or unsupported claims, (4) heavily penalize numerical hallucinations, (5) provide reasons emphasizing driver safety. The Confident AI G-Eval medical-domain template explicitly states "in regulated domains like healthcare, finance, or law, factual accuracy is critical ... faithfulness metrics in these settings should be designed to heavily penalize hallucinations, especially those that could affect high-stakes decisions" — racing safety is the direct analogue. `Deepeval`

LLM-as-judge with calibration: use a second model as the judge; per arXiv:2505.04847 (Hughes et al., 2025), "the highest effectiveness is achieved using the o3-mini-high judge, reaching a balanced accuracy of 84% and an F1-macro of 82.1%" on FaithBench, so plan an `o3-mini-high` class judge for the final paper-grade evaluation while using Granite Guardian 4.1 for the gate. Acknowledge judge bias (verbosity inflation, self-preference, position effects) explicitly. `arXiv`

4.4 Use Granite Guardian as both gate and reward model

Two distinct uses, both novel for IBM-stack judging:

- **Gate:** BYOC mode at the per-paragraph level — block paragraphs whose numerical claims do not match the projected forecast within tolerance.
- **Reward model:** Best-of-N sampling with Guardian as the scorer. Generate N=8 candidate paragraphs; Guardian picks the best. IBM reports Guardian 4.1 8B beats reward models up to 70B on JETTS (score 70.29) — cite in the paper.

4.5 Use IBM's own eval stack where possible

Wrap evaluation in **Unitxt** (IBM's open-source eval orchestration library, the backbone of FM-eval). Per IBM Research: "FM-eval evaluates models in a modular way ... we developed Unitxt, an open-source Python library that provides a consistent interface and methodology for defining datasets." Unitxt is on Hugging Face, integrates with `lm-eval-harness`, and gives your paper "we evaluated with IBM's own infrastructure" credibility. `IBM`

4.6 The benchmark you can actually publish

There is no public benchmark for "post-race AI race-engineer coaching faithfulness." Build one. Call it **APEX-Bench**:

- 50 laps × 5 corner-types × 3 car-classes × 3 COA-classes
- For each: raw 50 Hz telemetry, ground-truth coaching from a credentialed human race engineer (you can find one for 3–5 laps via the disabled-motorsport community)
- Metrics: accuracy / compliance / readiness / OOD + the coaching-faithfulness G-Eval rubric

Release it Apache 2.0 alongside the paper. This is the *single biggest* contribution-to-difficulty ratio in the project — a small curated benchmark in a new domain is a publication magnet.

4.7 Confirm vs. change — evaluation

Element	Verdict	Why
"Build a coaching report" — no metric	CHANGE	Add LIPS 4-axis + G-Eval faithfulness rubric
One ablation	CHANGE	Four ablation rows × four axes
No held-out OOD test	CHANGE	Held-out track × car × COA
No public benchmark	CHANGE	Release APEX-Bench

5. The "Shouldn't Be Possible" Architectural Moves

Five named, current (2025-2026), implementable-in-12-days moves. Ordered by integration cost in the build plan.

5.1 Granite 4.0 Nano 350M in the browser via WebGPU (Day 8-10)

`onnx-community/granite-4.0-350m-ONNX-web` + Transformers.js + WebGPU is shipping today. IBM's official Space `ibm-granite/Granite-4.0-Nano-WebGPU` demos in-browser tool calling. Per VentureBeat (April 2026), Granite 4.0 Nano "350M variants can run comfortably on a modern laptop CPU with 8-16 GB of RAM" and "the smallest ones can even run locally on your own web browser, as Joshua Lochner aka Xenova, creator of Transformer.js and a machine learning engineer at Hugging Face, wrote on the social network X." `Hugging Face + 2`

The "impossible" demo: Open one HTML file at the paddock with no internet, get a Granite-powered race recap on a phone or laptop. Zero server cost, zero latency, full offline operation. This is the single most visceral "wait, did a 2-person team build this?" moment in the demo video.

Implementation: A ~5 KB frontend chunk. Use it for the *recap mode* (terse summary), keep the full corner-by-corner coaching on the backend Granite 4.1 8B.

5.2 Activated LoRA (aLoRA) "race-engineer intrinsic" on vLLM (Day 6-9)

From Greenewald et al., arXiv:2504.12397: "We introduce Activated LoRA (aLoRA), a novel extension of the LoRA framework designed to only adapt the model's weights for tokens encountered *after* activation, allowing base model KV cache for prior context to be reused. By decoupling the adaptation process from the need to recompute the input's representation,

aLoRA facilitates instantaneous switching between specialized models (intrinsic) ... We demonstrate significant speedups for aLoRA vs LoRA on the state-of-the-art inference engine vLLM." The December 2025 follow-up (arXiv:2512.17910) presents "the first LLM serving engine, built upon the vLLM framework, that enables efficient adapter switching in multi-turn inference through prefix cache reuse between base and adapted models via activated LoRA." (arxiv + 2)

The move: Train a small aLoRA "race-engineer intrinsic" on top of Granite 4.1 8B Instruct using your APEX-Bench coaching set. Hot-swap mid-conversation without KV-cache recomputation. Greenewald et al. are IBM Research — perfect IBM-judge alignment, and you can cite it as your own systems contribution.

5.3 GEPA reflective prompt optimization for coaching (Day 7-10)

From Agrawal et al., arXiv:2507.19457: "GEPA (Genetic-Pareto) is a reflective optimizer ... that adaptively evolves textual components (such as prompts) of arbitrary systems. In addition to scalar scores returned by metrics, users can also provide GEPA with a text feedback to guide the optimization process." Public benchmarks (gepa-ai/gepa repo): "Prompt Triage: Structured Optimization for VLMs on Medical Imaging (Stanford) — median 53% relative improvement on 5 imaging tasks"; DSPy PAPILLON tutorial reports "GEPA optimize the PAPILLON program from a score of 77% to 86% after proposing just 1 new candidate." (DSPy + 2)

The move: Use GEPA via DSPy with a multi-objective metric (faithfulness G-Eval × physics-compliance violation rate). Run 50 generations on the APEX-Bench training split. Ship the optimized prompt as a checked-in artifact.

5.4 EAGLE-3 speculative decoding for the coaching latency target (Day 9-11)

From arXiv:2401.15077 and the EAGLE-3 release: per E2E Networks' summary, "EAGLE-3 achieves speedups between 2-6x depending on the model size and batch configuration. For Llama-3.1-8B at batch size 4, our benchmarks show 2.3x. The draft head adds under 5% parameters." Already supported in vLLM.

The move: Serve Granite 4.1 8B Instruct under vLLM with EAGLE-3 enabled. Target a sub-15 s coaching-report generation latency for a typical 12-corner session.

5.5 Tri-agent "Agent-as-Judge" critic panel (Day 5-8)

From Zhuge et al. (ICML 2025, arXiv:2410.10934): "In code-generation experiments, Agent-as-a-Judge agreed with human expert evaluations about 90% of the time—substantially outperforming the 70% agreement rate of previous LLM-as-a-Judge methods." MAJ-EVAL (arXiv:2507.21028) extends to multi-stakeholder NLG evaluation: "We propose MAJ-EVAL, an LLM-based multi-agent evaluation framework designed to simulate real-world multi-stakeholder-aligned NLG evaluation." (arXiv)

The move: Three Granite 4.1 agents in distinct roles (Physics-Critic, Pedagogy-Critic, Safety/Guardian), each with its own system prompt and own MCP toolset, debating each paragraph. Aggregate via simple debate-then-vote. This is the agentic-architecture flourish that makes the system look orders of magnitude more sophisticated than the underlying complexity.

5.6 Stretch: 1-lap test-time-training personalization

Hardt & Sun (ICLR 2024): test-time training "adapts its parameters at inference time using self-supervised objectives on the test stream itself." Apply it to the TTM head: fine-tune the last block in <60 s on the driver's warm-up laps before the race begins. This is the "the model gets to know you in a single warm-up" capability — surprising and reasonable for a 2-person team because TTM is sub-1M params.

6. Systems, Deployment, and Engineering Frontier

6.1 Deployment: not just Hugging Face Space + Vercel

Confirm: Vercel for the Next.js 16 / React 19 / TypeScript frontend is correct. Don't change it.

Recommend changing: A single Hugging Face Space is not enough for what you're shipping.

Recommended backend topology:

1. **Inference plane:** vLLM serving Granite 4.1 8B Instruct with **EAGLE-3 speculative decoding** + **PagedAttention** + **continuous batching**. Quantize to **FP8** for the 4060 dev environment (vLLM supports FP8, INT4, INT8, GPTQ, AWQ, GGUF). Benchmarks routinely show vLLM beats raw HuggingFace transformers ~3-4× on batched throughput (e.g., 3.38 s vs. 12.9 s at batch size 32 in the Ali Shafique offline benchmark). (Medium)
2. **Time-series plane:** PyTorch + `granite-tsfm`. Compile TTM and FlowState to **ONNX** for the inference path; keep PyTorch only for fine-tuning. ONNX gives you a 2-3× speedup and lets you ship the same artifact to the browser path (Transformers.js).
3. **Orchestration plane:** **LangGraph** server (FastAPI + Pydantic v2, which you already have) exposing every flow as an **MCP tool** via **IBM ContextForge** as the gateway.
4. **Observability plane:** **OpenTelemetry** traces from **ContextForge** → **Phoenix** (Arize) for free, beautiful agent-trace visualization. This is what makes the demo video *look* like a serious system.
5. **Edge plane:** the Granite 4.0 Nano WebGPU build, served as static assets from Vercel.
6. **Persistence plane:** Supabase (Postgres + storage) for telemetry uploads, per-driver session history, and the RAG vector store (pgvector) — single-vendor simplicity.

6.2 Latency engineering for the "60-second coaching loop"

Budget for a typical 12-corner session:

- File upload (CSV + PDF): 2 s
- Granite-Docling COA parse: 3 s
- Polyphase decomposition + TTM ensemble: 10 s
- CvxpyLayer projection (32 mini-sectors): 5 s (CvxpyLayer is the bottleneck; *cache the conic structure* — solve once per car class, not per request)
- Granite Embedding R2 retrieval: 1 s
- Granite 4.1 8B planning + drafting with EAGLE-3: 20 s
- Tri-agent critic pass: 12 s
- Mellea IVR repair (avg 0.5 repairs): 5 s
- Guardian final verdict: 2 s
- **Total: ~60 s**

If you blow the budget, kill the tri-agent panel for the live demo and run it asynchronously as a "deeper analysis arriving in 2 minutes" follow-up.

6.3 Reproducibility and open-source credibility

This is what makes the project survive after the deadline.

- **License:** Apache 2.0 across the board (everything in the Granite stack is already Apache 2.0).
- **Lockfiles:** `uv lock` for Python; `pnpm-lock.yaml` for the frontend. Pin everything.
- **Containerization:** A `Dockerfile` per plane (inference, orchestration, frontend). Compose for local dev. The Hugging Face Space uses the same Dockerfile.
- **Cryptographic signing:** Sign your model artifacts with `sigstore` (IBM cryptographically signs all Granite 4.1 artifacts as of 29 April 2026 — match this). `IBM`
- **Model card + system card:** Use IBM's Granite model-card template. Disclose the COA-handling privacy posture explicitly.
- **APEX-Bench:** Apache 2.0, on Hugging Face Datasets, with a `tasks.py` for `lm-eval-harness` integration via Unitxt.
- **Reproducibility seed:** every report runs with `seed=42` by default; deterministic CvxpyLayer (CVXPY uses ECOS deterministically).
- **CI:** GitHub Actions running the four-axis ablation on a small held-out set on every PR. This is what makes a workshop reviewer think "this is a serious project."
- **A demo Space + a code repo:** the Space runs the inference; the repo runs the training, ablation, and benchmarking.

6.4 Confirm vs. change — systems

Element	Verdict	Why
Next.js 16 + React 19 + TS on Vercel	CONFIRM	Correct
FastAPI + Pydantic v2 + cvxpylayers + torch	CONFIRM	Correct
Hugging Face Space deploy	CONFIRM + extend	Add vLLM + EAGLE-3 inference plane, ContextForge gateway, Phoenix observability
One backend service	CHANGE	Inference plane + orchestration plane + edge (WebGPU) plane
No quantization	CHANGE	FP8 on the 4060; ONNX for TTM/FlowState
No observability	CHANGE	OpenTelemetry → Phoenix
No public benchmark	CHANGE	Ship APEX-Bench
No browser/edge story	CHANGE	Ship Granite 4.0 Nano WebGPU recap

Recommendations — Staged 12-Day Plan with Thresholds

Stage 1 — Days 1-3: Spine

- Frontend scaffold (Next.js 16) and upload UI.
- Backend scaffold (FastAPI), Supabase wired, Docling + Granite-Docling parsing the COA PDF into DocTags.
- TTM r2.1 channel-mix decoder fine-tuning on a single car/track recorded session (you have one — the team is racers).
- **Threshold to proceed:** end-to-end CSV+PDF → JSON forecast in <30 s.

Stage 2 — Days 4-6: Physics + RAG

- CvxpyLayer Stage-1 + Stage-2 (from the prior physics pass).
- Granite Embedding R2 + LanceDB + cross-encoder reranker on a 100-document corpus.
- Granite 4.1 8B Instruct serving via vLLM (no EAGLE-3 yet).

- **Threshold to proceed:** end-to-end first coaching report generated with retrieved citations.

Stage 3 — Days 7-9: Agentic + Critic Loop

- LangGraph state machine.
- Tri-agent critic panel (Physics, Pedagogy, Guardian-BYOC).
- Mellea IVR loop with `loop_budget=3`.
- **Threshold to proceed:** G-Eval faithfulness ≥ 0.80 on a 10-lap dev set; max physical-compliance violation $\leq 1e-3$.

Stage 4 — Days 10-11: "Impossible" Demos + Evaluation

- Granite 4.0 Nano 350M WebGPU recap build.
- EAGLE-3 speculative decoding in vLLM.
- aLoRA "race-engineer intrinsic" — only attempt if Stage 3 is clean.
- LIPS 4-axis ablation on APEX-Bench dev split.
- **Threshold to proceed:** all four ablation rows have numbers; browser demo loads in <8 s.

Stage 5 — Day 12: Paper + Polish

- Workshop paper draft (4 pages + appendix) — sections built directly from the evaluation section above.
- Demo video.
- Model card + system card.
- APEX-Bench v0.1 release.

Benchmarks that should change the plan

- If LIPS axis "industrial readiness" misses 60 s: drop tri-agent critic to single Granite Guardian gate; keep tri-agent as offline batch.
- If G-Eval faithfulness < 0.70 : drop coaching language to numerical-only with templated prose; do not ship unfaithful free text.
- If aLoRA training does not converge by Day 9: drop it; keep the citation in the related-work section.
- If WebGPU build is >12 MB compressed: drop to the official IBM Space iframe.

Caveats

- TSFM benchmarks (GIFT-Eval, fev-bench, Chronos-2 paper) have known data-leakage

issues. Meyer et al. (arXiv:2510.13654, October 2025) report "documented cases of leakage between training and test sets have led to over 50% better test scores," and a controlled Moirai experiment with just 0.1% data leakage showed MAPE gains of 7.6–59 percentage points. Don't quote leaderboard rankings as gospel; report your own numbers on APEX-Bench.

- Some sources cited above are dated late 2025 / early 2026 — the EAGLE-3 speedup, the GEPA results, the Toto 2.0 release, the Granite 4.1 release. Where a result is from a vendor blog, the corresponding peer-reviewed paper may not yet exist. Mark these as "vendor-reported" in your paper.
- "Kinetic hallucination" is your team's term, not a standard term. It is good rhetoric but you should ground it in the literature on "physics-consistent forecasting" and "constraint-satisfaction in learned dynamics" — cite LIPS and the Dalal et al. safety-layer line of work.
- The Granite Guardian "reward model on JETTS outperforms 70B models" claim (score 70.29) is IBM-reported on a single benchmark; cite carefully, ideally alongside an independent run on your APEX-Bench set.
- Mellea is a recent IBM Research project; check that the `loop_budget` IVR pattern is in the released package before committing to it for the demo (it is in the public docs as of late 2025 but the API may move).
- The aLoRA + vLLM serving paper (arXiv:2512.17910) is a December 2025 preprint; the integration code may require building from source.
- Browser-side Granite 4.0 Nano is real today but quality degrades vs. the server 8B — keep the browser path to *recap mode* only, never the safety-critical coaching path.
- The CvxpyLayer is the *latency single-point-of-failure*; cache the conic structure per car class or the 60 s budget is at risk.
- The Chronos-2 "win rate" figure is specifically 90.7% on fev-bench (SQL metric) against the full field with the next-best TiRex at 80.8%; this is distinct from the head-to-head win rate against Chronos-Bolt, which is sometimes loosely referred to as ">90%." Use the 90.7% / 80.8% number in the paper for precision.